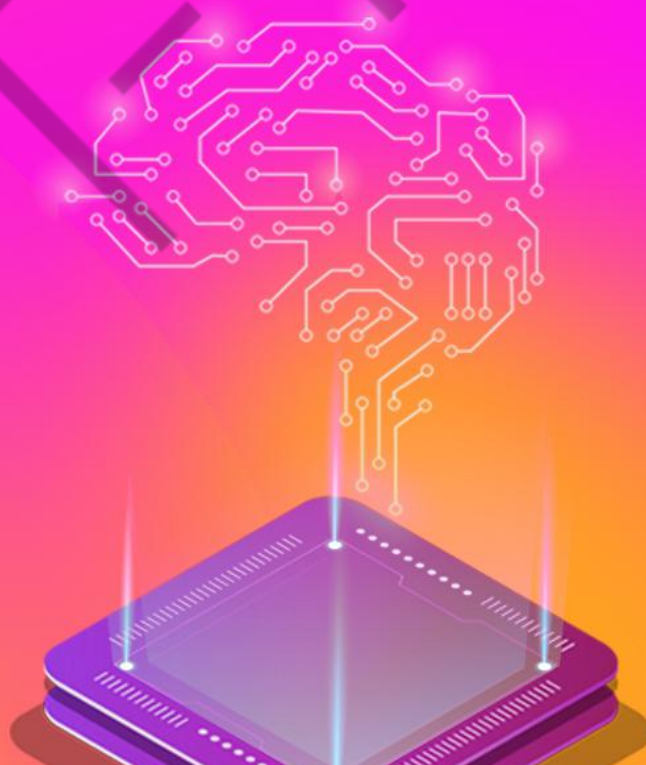




GESTÃO DE CLUSTERS,
SEGURANÇA QUÂNTICA E
COLABORAÇÃO MULTIAGENTE

APLICANDO INTELIGÊNCIA COM ML NO MUNDO FÍSICO



LISTA DE FIGURAS

Figura 1 - Pipeline de coleta, desenvolvimento, teste e implantação de modelos para execução em dispositivos de borda	9
Figura 2 - Estrutura hierárquica que separa o treinamento em nuvem da inferência na borda e em dispositivos TinyML	10
Figura 3 - Exemplo de dataset bruto contendo leituras do acelerômetro e rótulos associados	16
Figura 4 - Ranking das 20 principais features segundo o F-Score ANOVA	21
Figura 5 - Exemplo do dataset reduzido contendo as principais features	23
Figura 6 - Representação conceitual do hiperplano ótimo e dos vetores de suporte em um classificador SVM Linear	25
Figura 7 - Matriz de confusão e relatório de métricas do modelo SVM Linear aplicado ao conjunto de validação	30

LISTA DE QUADROS

Quadro 1 - Benefícios e limitações do processamento embarcado em IoT	12
--	----

EMBAR

LISTA DE TABELAS

Tabela 1 - Distribuição das classes nos conjuntos completo, de treino e de validação após a divisão estratificada	28
---	----

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Extração de features temporais e espectrais a partir de janelas do sinal de aceleração em projeto IoT com ESP32 e MPU6050.....	19
Código-fonte 2 - Cálculo do F-Score ANOVA e ranqueamento das features extraídas do acelerômetro para o problema de classificação repouso <i>versus</i> movimento	20
Código-fonte 3 - Seleção das principais features com base no F-Score	22
Código-fonte 4 - Divisão em dataset de treino e validação	28
Código-fonte 5 - Validação do modelo SVM Linear e geração da matriz de confusão	30
Código-fonte 6 - Extração dos coeficientes e bias do SVM Linear.....	32

SUMÁRIO

1 APLICANDO INTELIGÊNCIA COM ML NO MUNDO FÍSICO	7
1.1 Fundamentos do Machine Learning Embarcado	7
1.2 Conceito de aprendizado de máquina em dispositivos IoT	8
1.3 Diferenças entre treinamento em nuvem e inferência na borda	9
1.4 Benefícios e limitações do processamento embarcado	10
1.5 Aplicações práticas de Machine Learning embarcado	12
2 TREINAMENTO, VALIDAÇÃO E EXTRAÇÃO DO MODELO	14
2.1 Estrutura do dataset e pré-processamento dos dados	14
2.2 Extração de Features e Construção do Dataset de Treino	17
2.3 Seleção do Algoritmo de Aprendizado	24
2.4 Divisão entre Conjuntos de Treino e Teste	26
2.5 Validação do Modelo e Matriz de Confusão	28
2.6 Extração dos Coeficientes e Bias para o Firmware	31
2.7 Interpretação Matemática da Equação do Modelo	33
REFERÊNCIAS	35

1 APLICANDO INTELIGÊNCIA COM ML NO MUNDO FÍSICO

1.1 Fundamentos do Machine Learning Embarcado

O Machine Learning embarcado representa a integração entre técnicas de Inteligência Artificial e sistemas de computação utilizados em dispositivos de Internet das Coisas (IoT). Em vez de operar de forma puramente reativa, enviando dados para servidores remotos, os dispositivos passam a interpretar informações localmente e a tomar decisões de maneira autônoma. Com isso, sensores e atuadores deixam de ser elementos passivos e passam a desempenhar um papel ativo na análise de padrões e no acionamento de respostas em tempo real.

Esse avanço se tornou possível graças à evolução dos microcontroladores, que atualmente oferecem maior capacidade de processamento, conectividade integrada e baixo consumo de energia. A necessidade de reduzir a latência, preservar a privacidade dos dados e manter o funcionamento mesmo em ambientes com pouca conectividade reforçou o uso do processamento local. Em outras palavras, ao deslocar a inferência para a borda, os sistemas IoT tornam-se mais rápidos, eficientes e independentes de infraestrutura externa.

Nesse contexto surgiu o conceito de TinyML. TinyML consiste em um conjunto de técnicas que possibilitam a execução de modelos de aprendizado de máquina em microcontroladores muito limitados, frequentemente com apenas alguns kilobytes de memória. Isso inclui desde algoritmos clássicos até versões compactas de redes neurais. O resultado é a possibilidade de realizar tarefas avançadas de reconhecimento de padrões, análise de áudio e identificação de movimentos em dispositivos extremamente pequenos e econômicos.

O fluxo de desenvolvimento de Machine Learning embarcado envolve algumas etapas principais. Primeiro ocorre a coleta e rotulagem dos dados, etapa já concluída anteriormente. Em seguida, ocorre o treinamento do modelo em um ambiente mais robusto, normalmente utilizando Python e bibliotecas especializadas. Depois de validado, esse modelo é convertido para uma forma compacta, geralmente representado por equações matemáticas ou estruturas otimizadas para microcontroladores. A etapa final consiste na implementação desse modelo

diretamente no firmware, garantindo que a inferência seja executada continuamente durante a operação do dispositivo.

O resultado desse processo é um sistema IoT mais inteligente e eficiente. A execução local reduz o tráfego de dados, melhora o tempo de resposta e aumenta a confiabilidade, já que o dispositivo não depende de conexão constante com a internet para funcionar. Essa abordagem fortalece o desenvolvimento de aplicações avançadas em ambientes domésticos, urbanos, industriais e hospitalares, consolidando o Machine Learning embarcado como um dos pilares fundamentais da próxima geração de dispositivos inteligentes.

1.2 Conceito de aprendizado de máquina em dispositivos IoT

O aprendizado de máquina em dispositivos IoT permite que sensores e microcontroladores interpretem dados e tomem decisões diretamente na borda da rede. Em vez de enviar todos os dados para a nuvem, o dispositivo passa a executar modelos previamente treinados, o que reduz a latência, diminui o consumo de banda, aumenta a privacidade das informações e fortalece a autonomia do sistema.

Esse processo envolve quatro etapas principais. A primeira etapa consiste na coleta de dados, que são capturados continuamente pelos sensores embarcados. A segunda etapa corresponde ao desenvolvimento e treinamento do modelo em um ambiente computacional mais robusto. A terceira etapa envolve a avaliação do desempenho do modelo para garantir sua precisão. A etapa final consiste na implantação do modelo no dispositivo IoT, onde ocorrerá a inferência em tempo real.

A ilustração a seguir mostra esse fluxo completo, desde a coleta até o envio do modelo embarcado para execução no dispositivo. A figura destaca o ciclo Collect, Design, Test e Deploy, amplamente utilizado em materiais científicos sobre TinyML e Edge AI, representando de forma clara como o aprendizado de máquina se integra ao hardware de borda.

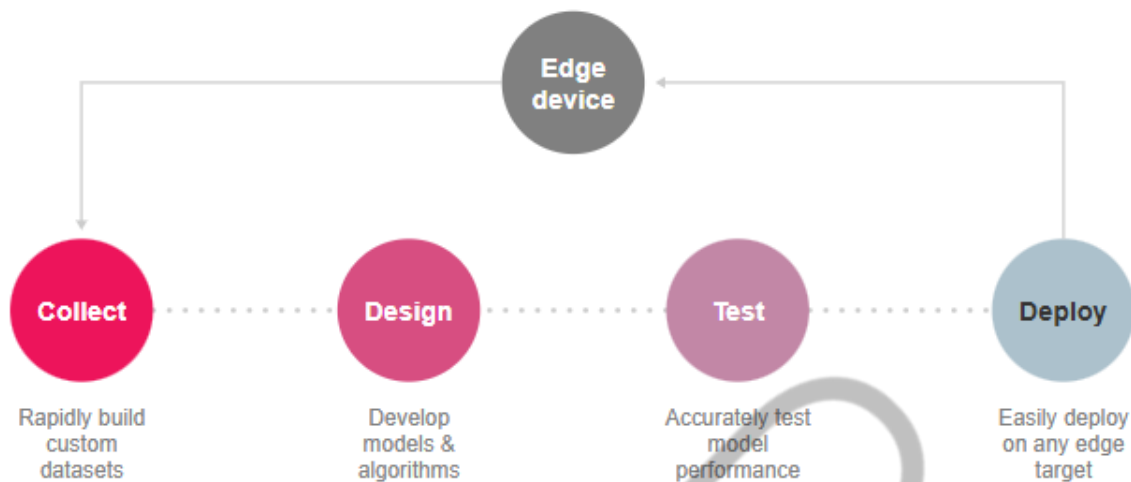


Figura 1 - Pipeline de coleta, desenvolvimento, teste e implantação de modelos para execução em dispositivos de borda

Fonte: Edge Impulse Documentation (2023), adaptado pelo autor (2025)

Essa representação visual serve como base para compreender como modelos treinados fora do dispositivo podem ser convertidos para versões compactas adequadas ao ambiente embarcado. As próximas subseções aprofundam as diferenças entre o treinamento em nuvem e a inferência local, além de abordar os benefícios e desafios dessa abordagem.

1.3 Diferenças entre treinamento em nuvem e inferência na borda

O processo de aprendizado de máquina em IoT é dividido em duas etapas principais. A primeira etapa é o treinamento, que ocorre em ambientes de alta capacidade computacional, como servidores ou plataformas em nuvem. Nessa fase, algoritmos processam grandes volumes de dados e ajustam parâmetros internos do modelo, atividade que exige recursos intensivos de hardware, como GPU, TPU ou clusters especializados.

A segunda etapa é a inferência, que corresponde ao uso do modelo treinado para realizar previsões a partir de novos dados. A inferência pode ser executada diretamente em dispositivos de borda, como microcontroladores ou sistemas embarcados. Isso reduz a latência, melhora a privacidade, diminui o tráfego de rede e permite que aplicações funcionem mesmo com conectividade limitada.

A figura a seguir apresenta uma visão clara dessa separação, mostrando como o treinamento se concentra na nuvem, enquanto a inferência é deslocada para a borda, incluindo dispositivos embarcados e plataformas TinyML.

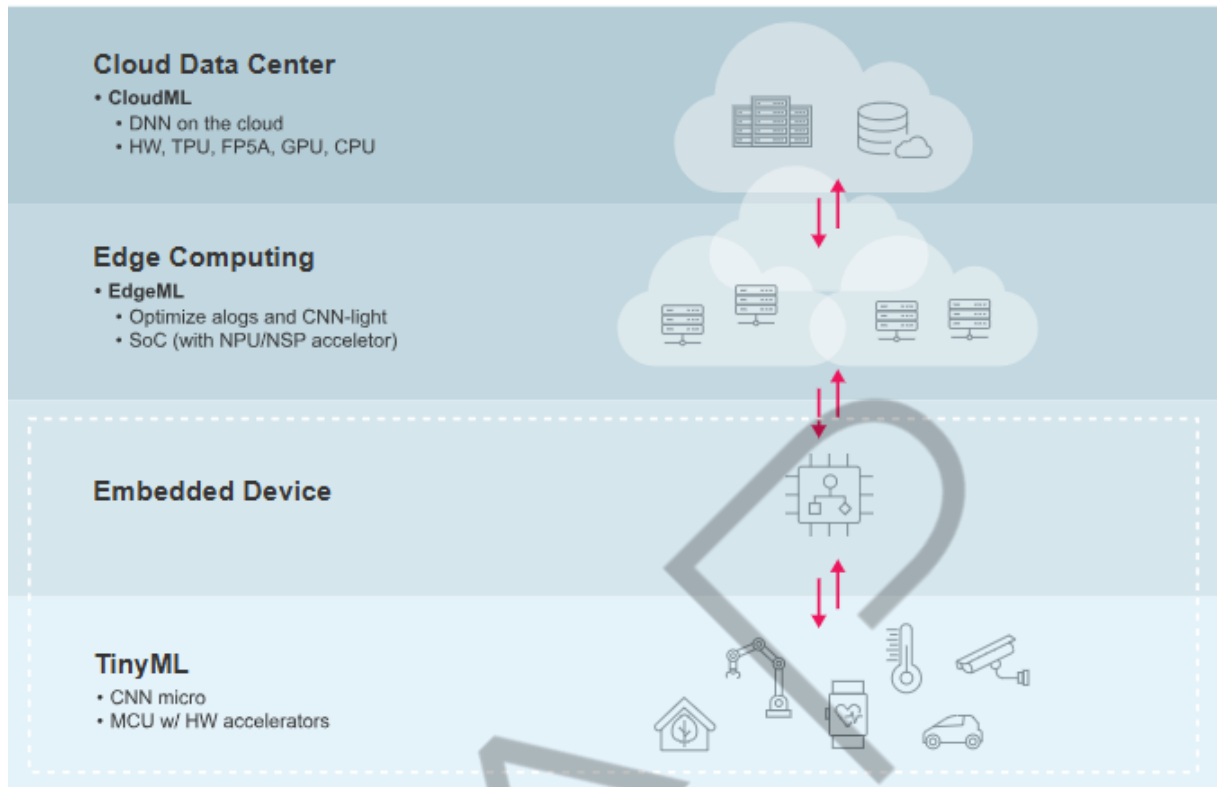


Figura 2 - Estrutura hierárquica que separa o treinamento em nuvem da inferência na borda e em dispositivos TinyML

Fonte: Kim *et al.* (2022), adaptado pelo autor (2025)

Essa divisão de responsabilidades possibilita o desenvolvimento de sistemas IoT inteligentes que aproveitam a capacidade computacional da nuvem sem abrir mão da autonomia e da eficiência da inferência local. Essa estratégia combina desempenho, privacidade e escalabilidade, tornando-se fundamental para aplicações modernas em sensores inteligentes, wearables, automação e sistemas industriais conectados.

1.4 Benefícios e limitações do processamento embarcado

O processamento embarcado em dispositivos IoT permite que decisões sejam tomadas diretamente no ponto de coleta dos dados, sem a necessidade de envio contínuo para servidores externos. Essa capacidade reduz a latência, melhora a privacidade e aumenta a autonomia dos dispositivos, tornando possível executar aplicações inteligentes mesmo em condições de conectividade limitada.

Entre os principais benefícios está a resposta imediata, resultado da proximidade entre o sensor e o processamento. Isso é essencial em aplicações que demandam tempo real, como detecção de anomalias, monitoramento industrial,

reconhecimento de movimentos e sistemas biomédicos portáteis. Outro benefício relevante é a economia de energia, já que grande parte do consumo em IoT vem da transmissão de dados. Quando apenas resultados importantes são enviados para a nuvem, o dispositivo se torna mais eficiente e pode operar por longos períodos utilizando baterias de baixa capacidade.

Além disso, o processamento local melhora a privacidade, pois reduz a circulação de dados sensíveis, como sinais fisiológicos, padrões comportamentais ou áudio. Isso facilita a conformidade com legislações de proteção de dados e aumenta a segurança das aplicações.

Por outro lado, existem limitações importantes. Microcontroladores possuem recursos muito menores que servidores em nuvem, como memória restrita, baixa capacidade de processamento e ausência de hardware especializado para operações matemáticas complexas. Como consequência, modelos embarcados precisam ser compactos e otimizados, o que pode impactar o desempenho preditivo. Também há desafios relacionados à manutenção, já que atualizações de modelos dependem de sincronização com servidores externos.

O quadro a seguir resume os principais benefícios e limitações do processamento embarcado em IoT.

Aspecto analisado	Benefícios do processamento embarcado	Limitações do processamento embarcado
Latência	Resposta imediata e previsível.	Limitada pela capacidade do microcontrolador.
Privacidade	Dados permanecem no dispositivo.	Modelos grandes não podem ser executados localmente.
Consumo de energia	Redução do tráfego de rede e maior autonomia.	Processamento intenso pode elevar o consumo local.
Conectividade	Funciona mesmo com internet instável ou inexistente.	Atualização de modelos exige infraestrutura externa.
Robustez	Menos dependente de falhas de comunicação.	Exige otimizações e ajustes específicos para cada hardware.
Tamanho dos modelos	Versões compactadas podem ser executadas.	Restrição severa de RAM, Flash e clock.
Escalabilidade	Facilmente replicável em diversos dispositivos.	Cada dispositivo precisa realizar seu próprio processamento.

Quadro 1 - Benefícios e limitações do processamento embarcado em IoT
Fonte: Elaborado pelo autor (2025)

O processamento embarcado equilibra autonomia e eficiência, mas requer atenção às limitações de hardware e à necessidade de modelos otimizados. É esse equilíbrio que define a escolha pela borda ou pela nuvem no desenvolvimento de soluções inteligentes em IoT.

1.5 Aplicações práticas de Machine Learning embarcado

O Machine Learning embarcado permite que sensores de baixo custo executem análises complexas diretamente no microcontrolador, tornando sistemas IoT mais rápidos, eficientes e autônomos. Em vez de depender continuamente da nuvem, o dispositivo realiza inferências localmente e transmite apenas informações relevantes, reduzindo consumo de banda e garantindo respostas imediatas. A seguir, são apresentados três exemplos representativos de aplicações amplamente utilizadas em soluções práticas.

O primeiro exemplo está em um projeto de identificação de buracos em pavimentos viários utilizando acelerômetro embarcado em veículos. Nesse tipo de solução, um microcontrolador recebe continuamente dados de aceleração enquanto o veículo se desloca pela via. As janelas de dados capturam picos e variações bruscas

associadas a impactos causados por buracos ou irregularidades do asfalto. A partir dessas janelas, o firmware calcula features como variação instantânea, energia do sinal e intensidade dos picos, e o modelo embarcado classifica os trechos como pavimento regular ou com presença de buracos. Com a inferência sendo realizada diretamente no dispositivo, é possível mapear a qualidade das vias sem necessidade de enviar todo o sinal bruto para a nuvem, o que reduz custos de comunicação e aumenta a autonomia da solução.

Um segundo exemplo importante está na manutenção preditiva de motores e máquinas rotativas, em que acelerômetros são utilizados para identificar comportamentos vibracionais anômalos. Equipamentos industriais apresentam assinaturas de vibração relativamente estáveis durante o funcionamento normal. Alterações nesses padrões podem revelar desgaste, desalinhamento, atrito excessivo ou falhas em rolamentos. O modelo embarcado interpreta essas mudanças em tempo real e identifica anomalias antes que a máquina apresente falhas críticas. Ao realizar a classificação diretamente no dispositivo instalado próximo ao motor, é possível monitorar ativos em ambientes remotos ou com conectividade limitada, garantindo detecção precoce com baixo custo energético.

O terceiro exemplo envolve detecção acústica embarcada para reconhecer eventos sonoros específicos em ambientes industriais ou de monitoramento, utilizando microfones de baixo consumo. O dispositivo capta o áudio ambiente e o microcontrolador extrai características, como espectro de frequências, energia por faixa ou coeficientes MFCC. O modelo embarcado pode identificar sons que indicam risco ou funcionamento anormal, como estalos mecânicos, ruídos de atrito, impactos repetitivos ou vazamentos de ar sob pressão. Como apenas o evento detectado é enviado para a rede, preserva-se a privacidade e evita-se o alto consumo de banda associado ao envio contínuo de áudio bruto.

Esses três casos demonstram como o Machine Learning embarcado transforma dispositivos simples em unidades inteligentes capazes de reconhecer padrões, detectar anomalias e agir de maneira autônoma. Essa abordagem reduz a dependência da nuvem, melhora a eficiência energética e aumenta a robustez do sistema em cenários reais. Na próxima seção, serão detalhadas as etapas de treinamento, validação e preparação do modelo para execução no microcontrolador.

2 TREINAMENTO, VALIDAÇÃO E EXTRAÇÃO DO MODELO

Uma vez que o dataset rotulado está disponível, inicia-se o processo de transformação desses dados em um modelo de aprendizado de máquina capaz de realizar inferências de forma confiável e eficiente. Este processo é essencial no desenvolvimento de sistemas IoT inteligentes, especialmente quando o objetivo é executar o modelo diretamente em microcontroladores de baixo consumo energético.

O treinamento de modelos para execução embarcada envolve uma série de etapas metodológicas que garantem não apenas a precisão da classificação, mas também a viabilidade do modelo dentro das restrições computacionais do dispositivo. Entre essas etapas estão a organização e preparação dos dados, a escolha do algoritmo mais adequado, a divisão dos dados em conjuntos de treino e teste, a avaliação rigorosa por métricas estatísticas e a seleção das características mais relevantes do sinal.

Após o treinamento e validação, o modelo precisa ser convertido em uma forma explícita e compacta, geralmente uma função matemática simples, cujos parâmetros são diretamente incorporados ao firmware. Essa extração é fundamental para possibilitar que o microcontrolador execute a decisão de forma rápida, determinística e com baixo consumo de energia.

Nesta seção, serão apresentadas todas as etapas necessárias para treinar um modelo leve e adequado para execução local, incluindo a preparação do dataset, a validação do desempenho e a extração final dos coeficientes que compõem o classificador embarcado. O objetivo é fornecer uma visão completa do ciclo de desenvolvimento de um modelo enxuto e eficiente, pronto para operar autonomamente em sistemas IoT.

2.1 Estrutura do dataset e pré-processamento dos dados

A construção de modelos supervisionados em sistemas IoT começa pela organização adequada dos dados coletados pelos sensores. Em aplicações envolvendo sinais inerciais, como detecção de movimento, vibração ou mudanças de orientação, um dataset rotulado é composto por amostras individuais que registram leituras brutas do sensor e o rótulo correspondente ao estado observado no instante da coleta. De modo geral, cada linha contém um timestamp que indica o momento da

leitura, valores numéricos relacionados ao comportamento físico do sistema e uma coluna de classe que permite ao algoritmo aprender a distinguir padrões. Essa estrutura é típica em diversas aplicações de IoT, como monitoramento de atividade humana, detecção de quedas, telemetria veicular e manutenção preditiva. Em cada caso, o dataset pode variar em número de colunas e sensores envolvidos, incluindo aceleração, rotação, campo magnético, temperatura ou sinais acústicos, mas mantém a mesma lógica central: registrar medições contínuas e associá-las a rótulos previamente definidos com base no fenômeno observado.

No caso de acelerômetros, é comum trabalhar com leituras triaxiais que capturam a dinâmica instantânea do sistema ao longo dos eixos x, y e z. Essas leituras, combinadas com o timestamp, permitem reconstruir o comportamento temporal do sinal e, posteriormente, aplicar métodos de pré-processamento e extração de características. Essa estrutura também facilita variações de projeto, permitindo que diferentes problemas sejam investigados sem alterar profundamente o formato dos dados. Por exemplo, é possível classificar movimentos específicos, identificar vibrações de motores, detectar desbalanceamentos mecânicos ou distinguir estados operacionais de máquinas rotativas apenas modificando os rótulos e as condições de coleta.

Neste capítulo, utilizaremos um dataset real obtido com um microcontrolador ESP32 conectado ao acelerômetro MPU6050, operando com taxa de amostragem aproximada de 50 Hz. Durante a coleta, o dispositivo foi submetido a períodos distintos de repouso e movimento, e cada amostra recebida foi rotulada de acordo com o estado observado no momento da medição. Assim, o dataset contém quatro elementos principais: timestamp, aceleração nos eixos ax, ay e az e a coluna label, que informa se o dispositivo estava parado ou se encontrava em movimento.

A imagem a seguir Figura 3 apresenta um trecho dessa base de dados, que servirá como ponto de partida para todas as etapas seguintes, incluindo pré-processamento, extração de features e treinamento do modelo. Essa visualização inicial permite compreender a natureza dos dados brutos antes de aplicarmos técnicas que transformarão as amostras individuais em representações adequadas para algoritmos de aprendizado de máquina.

timestamp	ax	ay	az	label
02:41:58	-0,151	-0,15	0,88	repouso
02:41:58	-0,088	-0,133	0,913	repouso
02:41:58	-0,113	-0,05	0,981	repouso
02:41:58	0,069	-0,025	1,141	repouso
02:41:58	0,021	-0,138	0,795	repouso
02:41:58	0,017	-0,196	0,907	repouso
02:41:58	0,169	-0,188	1,082	repouso
02:41:58	0,152	-0,187	1,216	repouso
02:41:58	0,18	-0,208	1,121	repouso
02:41:59	0,115	-0,201	1,135	repouso
02:41:59	0,074	-0,188	1,119	repouso
02:41:59	0,085	-0,194	1,059	repouso
02:41:59	-0,026	-0,223	0,981	repouso
02:41:59	0,069	-0,205	1,103	repouso
02:41:59	0,066	-0,171	1,159	repouso
02:41:59	0,014	-0,193	1,089	repouso
02:41:59	0,036	-0,196	1,116	repouso
02:41:59	0,029	-0,205	1,037	repouso
02:41:59	0,05	-0,185	1,123	repouso
02:41:59	-0,046	-0,213	1,062	repouso
02:41:58	-0,151	-0,15	0,88	movimento
02:41:58	-0,088	-0,133	0,913	movimento
02:41:58	-0,113	-0,05	0,981	movimento
02:41:58	0,069	-0,025	1,141	movimento
02:41:58	0,021	-0,138	0,795	movimento
02:41:58	0,017	-0,196	0,907	movimento
02:41:58	0,169	-0,188	1,082	movimento
02:41:58	0,152	-0,187	1,216	movimento
02:41:58	0,18	-0,208	1,121	movimento
02:41:59	0,115	-0,201	1,135	movimento
02:41:59	0,074	-0,188	1,119	movimento
02:41:59	0,085	-0,194	1,059	movimento

Figura 3 - Exemplo de dataset bruto contendo leituras do acelerômetro e rótulos associados
 Fonte: Elaborado pelo autor (2025)

A partir desta estrutura inicial, o próximo passo consiste em organizar essas amostras brutas de forma que se tornem úteis para um algoritmo de classificação. Isso envolve transformar séries contínuas de medições em segmentos interpretáveis, extrair estatísticas relevantes que descrevam cada trecho do sinal e preparar o dataset em um formato adequado para treinamento. As próximas subseções apresentam esse processo de maneira progressiva, conduzindo da representação bruta mostrada na figura anterior até a construção de um modelo otimizado para execução embarcada.

2.2 Extração de Features e Construção do Dataset de Treino

A etapa de extração de features tem como objetivo transformar as séries temporais brutas, geradas pelo acelerômetro embarcado no dispositivo IoT, em uma representação mais compacta e informativa para o modelo de aprendizado de máquina. Em vez de utilizar diretamente cada amostra do sinal, segmenta-se o fluxo de dados em janelas de tempo e, para cada janela, calcula-se medidas numéricas que resumem o comportamento do sinal naquele intervalo. Esse processo é especialmente importante em cenários de Machine Learning embarcado, pois permite reduzir o volume de dados, diminuir o custo computacional e facilitar a implementação do modelo em microcontroladores com recursos restritos.

De forma geral, as features podem ser extraídas no domínio do tempo ou da frequência. No **domínio do tempo**, são amplamente utilizadas medidas estatísticas como média, desvio-padrão, mínimo, máximo, amplitude peak to peak, energia e RMS, que descrevem a intensidade, a variabilidade e a distribuição da aceleração ao longo da janela. Outras medidas, como assimetria (skewness), curtose e número de cruzamentos por zero, ajudam a capturar a forma do sinal e a presença de oscilações ou picos. Já no **domínio da frequência**, aplica-se a Transformada Rápida de Fourier para obter a amplitude das primeiras componentes espectrais, que indicam como a energia do sinal se distribui em diferentes faixas de frequência. Em aplicações de IoT, a escolha das features deve equilibrar poder discriminativo e custo computacional, de modo que o conjunto final seja adequado tanto ao treinamento quanto à execução embarcada.

Na parte prática deste capítulo, a extração de features é realizada a partir do arquivo **dados_rotulados.csv**, que contém as amostras brutas de aceleração rotuladas como repouso ou movimento. O primeiro script em Python, apresentado no código-fonte a seguir, ilustra todo o processo de segmentação e cálculo das features. Inicialmente, são definidos o tamanho da janela (`WINDOW = 20` amostras), o número de componentes de frequência que serão extraídas pela FFT (`N_FFT = 5`) e os eixos do acelerômetro considerados na análise (`EIXOS = ["ax", "ay", "az"]`). Em seguida, o dataset é lido e percorrido em blocos sequenciais de 20 amostras. Para cada bloco, a função `extrair_features_bloco` verifica se todas as amostras pertencem à mesma classe; apenas nesses casos o segmento é aceito, garantindo que cada janela represente um único estado do sistema.

Dentro dessa função, são calculadas, para cada eixo, as principais features no domínio do tempo e da frequência. No domínio do tempo, são obtidas média (mean), desvio padrão (std), mínimo (min), máximo (max), amplitude peak to peak (ptp), energia (energy), RMS (rms), assimetria (skew), curtose (kurtosis) e número de cruzamentos por zero (zero_crossings). Em seguida, a FFT é aplicada à série associada a cada eixo, e as amplitudes das cinco primeiras componentes de frequência são armazenadas como `fft_1` até `fft_5`. Cada conjunto de features é associado a um identificador de segmento, ao timestamp inicial da janela e ao rótulo correspondente. Ao final do processamento, todas as janelas válidas são reunidas em um novo DataFrame e salvas no arquivo `dados_rotulados_Todas_Features.csv`, que passa a representar o dataset de treino em nível de janelas.

```
import pandas as pd
import numpy as np
from scipy.stats import skew, kurtosis
from scipy.fft import fft

arquivo_entrada = "dados_rotulados.csv"
arquivo_saida = "dados_rotulados_Todas_Features.csv"

WINDOW = 20
N_FFT = 5
EIXOS = ["ax", "ay", "az"]

df = pd.read_csv(arquivo_entrada).reset_index(drop=True)

def extrair_features_bloco(bloco, segmento_id):
    feats = {
        "segmento_id": segmento_id,
        "timestamp_inicio": bloco["timestamp"].iloc[0],
        "label": bloco["label"].iloc[0]
    }
    for eixo in EIXOS:
        serie = bloco[eixo].values
        pref = eixo
        feats[f"{pref}_mean"] = np.mean(serie)
        feats[f"{pref}_std"] = np.std(serie)
        feats[f"{pref}_min"] = np.min(serie)
        feats[f"{pref}_max"] = np.max(serie)
        feats[f"{pref}_ptp"] = np.ptp(serie)
        feats[f"{pref}_energy"] = np.sum(serie ** 2)
        feats[f"{pref}_rms"] = np.sqrt(np.mean(serie
** 2))

        feats[f"{pref}_skew"] = skew(serie)
        feats[f"{pref}_kurtosis"] = kurtosis(serie)
```

```
        feats[f"{pref}_zero_crossings"] = ((serie[:-1] *
serie[1:]) < 0).sum()
        fft_vals = np.abs(fft(serie))[1:N_FFT + 1]
        for i, val in enumerate(fft_vals, start=1):
            feats[f"{pref}_fft_{i}"] = val
    return feats

lista_features = []
n_segmentos = len(df) // WINDOW

for seg in range(n_segmentos):
    ini = seg * WINDOW
    fim = ini + WINDOW
    bloco = df.iloc[ini:fim]
    if bloco["label"].nunique() != 1:
        continue
    lista_features.append(extrair_features_bloco(bloco,
seg + 1))

df_features = pd.DataFrame(lista_features)
df_features.to_csv(arquivo_saida, index=False)
print(df_features.shape)
```

Código-fonte 1 - Extração de features temporais e espectrais a partir de janelas do sinal de aceleração em projeto IoT com ESP32 e MPU6050

Fonte: Elaborado pelo autor (2025)

Com o dataset enriquecido por dezenas de features numéricas, a etapa seguinte consiste em avaliar quais delas são mais relevantes para distinguir as classes repouso e movimento. Para isso, utiliza-se o teste ANOVA via F-Score, disponível na função `f_classif` da biblioteca `scikit-learn`. O código-fonte a seguir mostra como esse procedimento é implementado.

Primeiro, o arquivo `dados_rotulados_Todas_Features.csv` é carregado, e são selecionadas apenas as colunas numéricas, excluindo identificadores e o rótulo. Em seguida, `f_classif` calcula, para cada feature, um F-Score que quantifica o quanto a distribuição daquela variável difere entre as duas classes. Quanto maior o F-Score, maior o poder discriminativo da feature. Os resultados são organizados em um DataFrame ordenado, e as 20 principais features são utilizadas para construir um gráfico de barras horizontais, que permite visualizar rapidamente quais características se destacam.

```
import numpy as np
import matplotlib.pyplot as plt

# df_scores já existe; aqui só pegamos o top 20
df_top20 = df_scores.head(20)

plt.figure(figsize=(14, 7))

# Cria uma lista de cores usando o colormap viridis
cores = plt.cm.viridis(np.linspace(0, 1, len(df_top20)))

plt.barh(
    df_top20["feature"],
    df_top20["f_score"],
    color=cores
)

plt.gca().invert_yaxis() # deixa a feature mais
importante em cima
plt.title("Top 20 Features por F-Score (ANOVA)",
fontsize=16)
plt.xlabel("F-Score", fontsize=12)
plt.ylabel("Feature", fontsize=12)
plt.tight_layout()
plt.show()
```

Código-fonte 2 - Cálculo do F-Score ANOVA e ranqueamento das features extraídas do acelerômetro para o problema de classificação repouso *versus* movimento
Fonte: Elaborado pelo autor (2025)

A figura a seguir Figura 4 apresenta o gráfico produzido por esse script, em que as 20 features mais importantes estão ordenadas do maior para o menor F-Score. Observa-se que as features associadas ao eixo z, como `az_std`, `az_ptp`, `az_fft_5`, `az_max`, `az_rms` e `az_energy`, aparecem no topo do ranking, indicando que esse eixo concentra a maior parte da variação relevante para diferenciar repouso de movimento no cenário estudado. Essa visualização auxilia tanto na compreensão intuitiva do problema quanto na justificativa técnica para a escolha do conjunto reduzido de features que será utilizado nas etapas seguintes de treinamento e implementação embarcada.

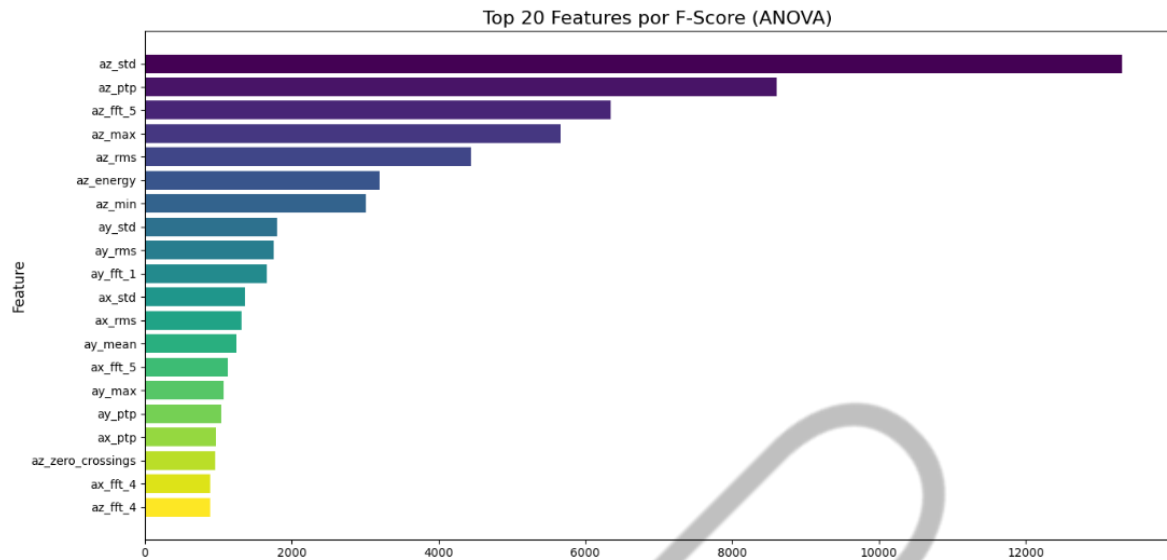


Figura 4 - Ranking das 20 principais features segundo o F-Score ANOVA
Fonte: Elaborado pelo autor (2025)

Com o ranking de importância em mãos, o próximo passo é selecionar automaticamente um subconjunto enxuto de features para compor o dataset final de treino, que servirá de entrada para o modelo SVM linear. O código-fonte “[Seleção das principais features com base no F-Score](#)” emprega a classe `SelectKBest`, também baseada no F-Score ANOVA, para escolher as sete melhores features.

O script lê novamente o arquivo com todas as features, separa as colunas numéricas e a coluna de rótulos, ajusta o seletor para $k = 7$ e treina o mecanismo de seleção. Em seguida, constrói-se um `DataFrame` com os valores de F-Score associados a cada feature, ordenando os resultados de forma decrescente. As sete primeiras são então listadas e utilizadas para criar um novo `DataFrame` reduzido, que contém apenas essas colunas e a coluna `label`. Esse dataset condensado é salvo em `dados_rotulados_Principais_Features.csv` e será utilizado diretamente no treinamento do modelo.

```
import pandas as pd
from sklearn.feature_selection import SelectKBest,
f_classif
arquivo_features = "dados_rotulados_Todas_Features.csv"
saida_principais =
"dados_rotulados_Principais_Features.csv"

df = pd.read_csv(arquivo_features)

X_full = df.drop(columns=["segmento_id",
"timestamp_inicio", "label"], errors="ignore")
X = X_full.select_dtypes(include=["number"])
y = df["label"]

k = 7
selector = SelectKBest(score_func=f_classif, k=k)
selector.fit(X, y)

scores_df = pd.DataFrame({
    "feature": X.columns,
    "f_score": selector.scores_
}).sort_values(by="f_score", ascending=False)

print(scores_df.head(k).to_string(index=False))

features_selecionadas =
scores_df["feature"].head(k).tolist()
print("Features selecionadas:", features_selecionadas)

df_reduzido = df[features_selecionadas + ["label"]]
df_reduzido.to_csv(saida_principais, index=False)
print(df_reduzido.shape)
```

Código-fonte 3 - Seleção das principais features com base no F-Score
Fonte: Elaborado pelo autor (2025)

A figura a seguir Figura 5 ilustra um trecho do dataset reduzido gerado por esse processo, no qual cada linha representa uma janela do sinal descrita pelas principais features do eixo z selecionadas pelo método ANOVA, acompanhadas do respectivo rótulo de classe. Observa-se que o conjunto final é compacto, composto apenas por poucas colunas, mas ainda assim preserva o poder discriminativo necessário para separar repouso e movimento. Esse equilíbrio entre simplicidade e desempenho é essencial para viabilizar a implementação do modelo em dispositivos embarcados, tema que será aprofundado nas subseções seguintes.

az_std	az_ptp	az_fft_5	az_max	az_rms	az_energy	az_min	label
0,010806827	0,037	0,036235342	1,021	1,001308319	20,052367	0,984	repouso
1,65468807	6,938	14,00303431	4,046	2,006147826	80,492582	-2,892	movimento
0,010229736	0,036	0,021931712	1,006	0,989102902	19,566491	0,97	repouso
1,629775011	7,349	13,91462917	4,996	1,921188213	73,819283	-2,353	movimento
0,009880663	0,041	0,008062258	0,96	0,936702114	17,548217	0,919	repouso
1,799100872	7,793	16,02394124	5,108	2,046049059	83,726335	-2,685	movimento
0,016773416	0,053	0,070007142	0,917	0,8876085	15,756977	0,864	repouso
1,783252009	8	15,37271859	5,108	2,031120934	82,509045	-2,892	movimento
0,018237599	0,079	0,056	0,96	0,932878288	17,405238	0,881	repouso
1,818477745	8	15,95279041	5,108	2,01817031	81,460228	-2,892	movimento
0,03149111	0,133	0,187066833	0,766	0,726782567	10,564258	0,633	repouso
1,648810689	6,788	10,80745322	4,072	1,865781941	69,622845	-2,716	movimento
0,017798595	0,064	0,130138388	0,85	0,820293118	13,457616	0,786	repouso
1,726161692	8	14,92045874	5,108	1,967547268	77,424845	-2,892	movimento
0,028308788	0,117	0,223805719	0,943	0,878206382	15,424929	0,826	repouso
1,94547234	8	16,96879292	5,108	2,147929177	92,271995	-2,892	movimento
0,017675972	0,068	0,130230565	0,881	0,848784071	14,408688	0,813	repouso
1,707083779	8	14,70274889	5,108	1,943902274	75,575121	-2,892	movimento
0,020609221	0,077	0,061400326	0,872	0,842852004	14,20799	0,795	repouso
1,649754273	7,822	13,6750336	4,93	1,918979208	73,649624	-2,892	movimento
0,03432124	0,154	0,192605815	0,933	0,844148002	14,251717	0,779	repouso
1,715228203	8	15,34705086	5,108	1,97144551	77,731948	-2,892	movimento
0,023890322	0,099	0,039962482	0,847	0,80830313	13,067079	0,748	repouso
1,784985246	7,786	14,28613121	4,894	2,033360802	82,691123	-2,892	movimento

Figura 5 - Exemplo do dataset reduzido contendo as principais features

Fonte: Elaboração do autor (2025)

A partir desse ponto, concluímos a transformação completa do sinal bruto em um conjunto estruturado, significativo e otimizado de informações para o aprendizado de máquina. O processo iniciado com a coleta dos dados do acelerômetro, passando pela segmentação em janelas, extração de features no domínio do tempo e da frequência, avaliação estatística via F-Score e redução inteligente das variáveis, resulta em um dataset compacto e altamente informativo. Esse dataset reduzido representa a base sobre a qual o modelo de classificação será treinado, garantindo que apenas as características com maior poder discriminativo sejam utilizadas. Essa estratégia melhora a eficiência computacional, reduz o risco de sobreajuste e prepara o terreno para que o modelo final seja convertido em uma equação simples, adequada para execução direta no microcontrolador.

Na próxima subseção, discutiremos como selecionar o algoritmo de aprendizado mais apropriado, dando sequência ao fluxo completo de desenvolvimento de um sistema de reconhecimento de movimento embarcado.

2.3 Seleção do Algoritmo de Aprendizado

A seleção do algoritmo de aprendizado é um passo decisivo no desenvolvimento de sistemas inteligentes embarcados. A escolha adequada depende diretamente das limitações de hardware, do formato das features e da complexidade do problema. Em dispositivos IoT com baixa capacidade de processamento, como microcontroladores da família ESP32, é essencial adotar modelos que sejam ao mesmo tempo leves, interpretáveis e eficientes em tempo de execução.

Diversas classes de algoritmos poderiam ser utilizadas para um problema de classificação binária como o apresentado neste projeto. Entre as alternativas mais comuns encontram-se:

- **Árvores de decisão e Random Forest**

Modelos de fácil interpretação, porém, costumam gerar estruturas extensas, com grande número de nós. Isso pode demandar mais memória e aumentar a latência durante a inferência em sistemas embarcados.

- **k-NN (k-Nearest Neighbors)**

Apesar de simples, o k-NN não é recomendado para implantação embarcada, pois exige armazenar todo o dataset e calcular distâncias a cada nova amostra, o que demanda mais RAM e energia.

- **Regressão Logística**

É leve e eficiente, porém pode apresentar desempenho limitado quando as fronteiras entre classes não são bem definidas linearmente.

- **Redes neurais artificiais (ANN / MLP)**

Embora flexíveis e poderosas, redes neurais possuem maior quantidade de parâmetros, demandam mais memória e apresentam maior custo computacional, o que pode ser inadequado para dispositivos de baixa potência.

Diante dessas alternativas, destaca-se como opção especialmente adequada para IoT o **Support Vector Machine (SVM)** com kernel linear.

O SVM Linear é um algoritmo supervisionado que busca separar classes por meio de um hiperplano que maximize a margem entre elas. Com isso, produz modelos extremamente compactos e estáveis, que podem ser representados com poucos

parâmetros e executados rapidamente. Essa compactidade o torna ideal para microcontroladores, onde memória e processamento são elementos limitados.

A Figura 6 a seguir ilustra a separação entre classes realizada por um SVM Linear, identificando hiperplanos, margens e vetores de suporte.

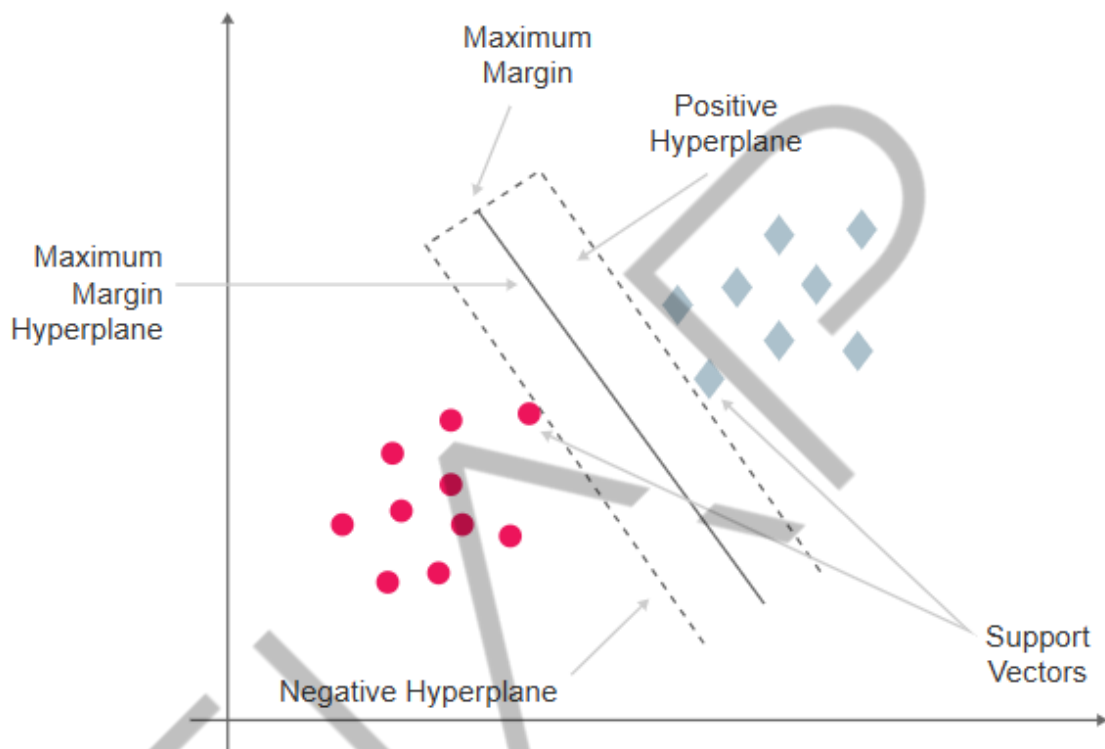


Figura 6 - Representação conceitual do hiperplano ótimo e dos vetores de suporte em um classificador SVM Linear

Fonte: Reddy (2024), adaptado pelo autor (2025)

A simplicidade estrutural do SVM Linear também facilita sua posterior conversão para firmware, já que o modelo resultante depende apenas de coeficientes numéricos e de uma operação matemática elementar. Isso reduz a complexidade da implementação embarcada e garante previsibilidade temporal, algo essencial em aplicações de reconhecimento de movimentos, monitoramento de ativos e sistemas responsivos em tempo real.

Embora neste capítulo o modelo escolhido tenha sido o SVM Linear, é importante mencionar que arquiteturas mais avançadas, como redes neurais executadas por meio do TensorFlow Lite Micro (TFLM), também são amplamente utilizadas em aplicações de aprendizado embarcado. O uso de TFLM, contudo, é mais indicado em cenários que exigem a extração automática de padrões complexos, como

reconhecimento de gestos mais variados, detecção de sons específicos ou classificação de imagens.

Para o problema tratado neste capítulo, que envolve apenas duas classes e um conjunto reduzido de features manualmente extraídas, o SVM Linear oferece uma solução mais simples, leve e eficiente, evitando o custo computacional e de memória associado a modelos neurais. Dessa forma, a escolha do SVM se justifica pelo equilíbrio entre desempenho, interpretabilidade e facilidade de implementação no ESP32, enquanto o TensorFlow Lite Micro permanece como alternativa interessante para projetos futuros que demandem modelos mais sofisticados.

Ao finalizar esta subseção, fica claro que, embora existam diversas alternativas de algoritmos possíveis, o SVM Linear oferece um bom equilíbrio entre desempenho, robustez e eficiência computacional. Assim, ele se torna uma opção adequada para o modelo de classificação embarcado que será desenvolvido nas próximas etapas do capítulo. A subseção seguinte abordará como organizar o dataset em conjuntos de treino e teste, preparando o ambiente para o treinamento efetivo do modelo.

2.4 Divisão entre Conjuntos de Treino e Teste

Depois de selecionar as principais features e estruturar o dataset em nível de janelas, o passo seguinte consiste em dividir os dados em dois subconjuntos com finalidades distintas. O primeiro, chamado **conjunto de treino**, é utilizado para ajustar os parâmetros do modelo de aprendizado de máquina. O segundo, chamado **conjunto de teste ou validação**, é reservado exclusivamente para avaliar o desempenho do modelo em dados que não foram vistos durante a fase de treinamento. Essa separação é fundamental para estimar a capacidade de generalização do modelo e evitar conclusões otimistas demais a partir de um ajuste realizado apenas sobre o conjunto completo.

De forma geral, a divisão entre treino e teste pode seguir diferentes proporções, como 70% para treino e 30% para teste, 80% para treino e 20% para teste, entre outras possibilidades. A escolha depende tanto do tamanho do dataset quanto da complexidade do problema. Em conjuntos de dados muito pequenos, reservar uma parcela excessiva para teste pode prejudicar o aprendizado do modelo; por outro lado, usar quase todos os dados para treino e deixar poucos exemplos para teste torna a avaliação pouco confiável. Ainda, em aplicações com séries temporais, pode ser

necessário preservar a ordem dos dados, utilizando os primeiros registros para treino e os mais recentes para teste, de modo a refletir um cenário mais próximo da operação real.

No contexto deste capítulo, o dataset reduzido foi inicialmente composto por 80 janelas, cada uma descrita por sete features e um rótulo de classe. As classes repouso e movimento estão perfeitamente balanceadas, com 40 exemplos para cada uma. A partir desse conjunto, adotou-se a divisão de 70% para treino e 30% para validação, mantendo a estratificação das classes. A estratificação garante que a proporção de exemplos de cada classe seja preservada em ambos os subconjuntos, evitando que um deles fique dominado por apenas uma classe e comprometendo a avaliação do modelo.

Na prática, a divisão foi realizada utilizando a função `train_test_split` da biblioteca `scikit-learn`, com a opção `stratify=y` para preservar o balanceamento entre as classes. O dataset de treino resultante contém 56 janelas, sendo 28 rotuladas como repouso e 28 como movimento. O dataset de validação contém 24 janelas, com 12 exemplos de repouso e 12 de movimento.

```
import pandas as pd
from sklearn.model_selection import train_test_split

arquivo_entrada = "dados_rotulados_Principais_Features.csv"
df = pd.read_csv(arquivo_entrada)

X = df.drop(columns=["label"], errors="ignore")
y = df["label"]

X_train, X_val, y_train, y_val = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=42,
    stratify=y
)

df_treino = X_train.copy()
df_treino["label"] = y_train.values

df_validacao = X_val.copy()
df_validacao["label"] = y_val.values
```

```
arquivo_treino = "dados_rotulados_Principais_Features_Treino.csv"
arquivo_validacao = "dados_rotulados_Principais_Features_Validacao.csv"

df_treino.to_csv(arquivo_treino, index=False)
df_validacao.to_csv(arquivo_validacao, index=False)
```

Código-fonte 4 - Divisão em dataset de treino e validação

Fonte: Elaborado pelo autor (2025)

Essa configuração mantém a simetria entre as classes em ambos os conjuntos e fornece uma quantidade adequada de exemplos para ajuste e avaliação do modelo. A tabela a seguir resume a distribuição das classes antes e depois da divisão.

Conjunto	Repouso	Movimento	Total	Porcentagem
Completo	40	40	80	100%
Treino	28	28	56	70%
Validação	12	12	24	30%

Tabela 1 - Distribuição das classes nos conjuntos completo, de treino e de validação após a divisão estratificada

Fonte: Elaborado pelo autor (2025)

Além da separação lógica dos dados, os subconjuntos foram salvos em arquivos distintos, denominados `dados_rotulados_Principais_Features_Treino.csv` e `dados_rotulados_Principais_Features_Validacao.csv`. Esses arquivos servirão como base para as próximas etapas do capítulo. O conjunto de treino será utilizado para ajustar o classificador SVM Linear, enquanto o conjunto de validação será empregado para medir o desempenho do modelo por meio de métricas como acurácia e matriz de confusão. Dessa forma, o fluxo de desenvolvimento do modelo se mantém organizado, reproduzível e alinhado às boas práticas de aprendizado de máquina.

2.5 Validação do Modelo e Matriz de Confusão

Depois de treinar um modelo de classificação, o passo seguinte consiste em validar seu desempenho em um conjunto de dados que não foi utilizado durante o treinamento. O objetivo é estimar a capacidade de generalização do modelo, isto é, verificar se ele consegue classificar corretamente novas amostras adquiridas pelo sistema IoT. Em problemas supervisionados, a validação normalmente é feita a partir

de métricas como acurácia, precisão, recall e F1-Score, calculadas com base nas predições do modelo e nos rótulos reais do conjunto de validação.

Uma forma bastante didática de organizar essas informações é por meio da matriz de confusão. Para um problema binário, como a distinção entre repouso e movimento, a matriz de confusão é uma tabela 2x2 que mostra quantas amostras de cada classe foram classificadas corretamente e quantas foram confundidas com a outra classe. Os elementos da diagonal principal representam acertos, enquanto os elementos fora da diagonal indicam erros de classificação. A partir dessa matriz, é possível derivar as demais métricas de desempenho, compreendendo não apenas quantas amostras o modelo acerta, mas também como se comporta em cada classe específica.

No contexto deste capítulo, utilizamos o conjunto de treino com 56 janelas (28 de repouso e 28 de movimento) para ajustar um classificador SVM Linear, e o conjunto de validação com 24 janelas (12 de cada classe) para avaliar seu desempenho. O processo de validação foi implementado em Python, carregando os arquivos de treino e validação, treinando o modelo e, em seguida, aplicando o classificador sobre o conjunto de validação para obter as predições. O código-fonte a seguir apresenta o script utilizado nessa etapa.

```
import pandas as pd
from sklearn.svm import SVC
from sklearn.metrics import confusion_matrix,
classification_report

arquivo_treino =
"dados_rotulados_Principais_Features_Treino.csv"
arquivo_val =
"dados_rotulados_Principais_Features_Validacao.csv"

df_treino = pd.read_csv(arquivo_treino)
df_val = pd.read_csv(arquivo_val)

X_train = df_treino.drop(columns=["label"])
y_train = df_treino["label"]

X_val = df_val.drop(columns=["label"])
y_val = df_val["label"]

modelo = SVC(kernel="linear")
modelo.fit(X_train, y_train)
```

```
y_pred = modelo.predict(X_val)

matriz = confusion_matrix(y_val, y_pred)
print("Matriz de Confusão:")
print(matriz)

print("\nRelatório de Classificação:")
print(classification_report(y_val, y_pred))
```

Código-fonte 5 - Validação do modelo SVM Linear e geração da matriz de confusão

Fonte: Elaborado pelo autor (2025)

A execução desse código gera como saída a matriz de confusão e o relatório de métricas apresentados na figura a seguir. Observa-se que todas as 12 amostras da classe movimento foram classificadas corretamente como movimento e todas as 12 amostras da classe repouso foram classificadas corretamente como repouso. Como consequência, a acurácia global é igual a 1,00 e os valores de precisão, recall e F1-Score também são iguais a 1,00 para ambas as classes.

Matriz de Confusão:

```
[[12  0]
 [ 0 12]]
```

Relatório de Classificação:

	precision	recall	f1-score	support
movimento	1.00	1.00	1.00	12
repouso	1.00	1.00	1.00	12
accuracy			1.00	24
macro avg	1.00	1.00	1.00	24
weighted avg	1.00	1.00	1.00	24

Figura 7 - Matriz de confusão e relatório de métricas do modelo SVM Linear aplicado ao conjunto de validação

Fonte: Elaborado pelo autor (2025)

Esses resultados indicam que, para o conjunto de dados utilizado, o modelo SVM Linear conseguiu separar corretamente as janelas de repouso e de movimento no conjunto avaliado, o que reforça a adequação das features extraídas e a adequação do algoritmo escolhido ao problema. Em aplicações reais, é importante lembrar que desempenhos tão elevados nem sempre serão observados, especialmente na presença de ruídos, movimentos intermediários ou variações nas condições de operação. Da mesma forma, é fundamental destacar que modelos supervisionados tendem a se beneficiar de conjuntos de dados maiores e mais

diversificados, que permitam capturar diferentes padrões do sinal e reduzir o risco de sobreajuste.

No presente capítulo, entretanto, a quantidade de dados rotulados foi mantida reduzida devido ao propósito exclusivamente acadêmico e demonstrativo. O objetivo é demonstrar, de maneira direta, cada etapa do pipeline de aprendizado de máquina aplicado a dispositivos IoT, desde a extração de features até a validação do modelo. Ainda assim, a validação realizada mostra que o fluxo adotado de extração de features, divisão estratificada dos dados e treinamento do SVM gera um classificador adequado ao propósito demonstrativo, pronto para ter suas features refinadas e seus coeficientes extraídos para implementação embarcada nas próximas subseções.

Além disso, é importante salientar que o tamanho reduzido do dataset influencia diretamente o comportamento do modelo durante o treinamento e a validação. Com poucos exemplos, especialmente quando bem separados entre as classes, o SVM tende a encontrar fronteiras de decisão muito precisas e com baixa variabilidade, o que pode resultar em desempenho aparentemente sem erros no conjunto utilizado. Contudo, esse cenário não necessariamente reflete a realidade de aplicações IoT, nas quais há ruídos no sensor, padrões de movimento mais complexos, diferenças entre dispositivos e condições ambientais variáveis.

Quanto maior a diversidade e a quantidade de dados, maior a capacidade do modelo de generalizar para situações novas, evitando o sobreajuste e garantindo respostas mais robustas no ambiente embarcado. Por isso, embora o resultado obtido seja adequado para fins didáticos, reforça-se que aplicações reais exigem um processo contínuo de coleta, expansão e validação do dataset.

2.6 Extração dos Coeficientes e Bias para o Firmware

Após a validação do modelo SVM Linear, o próximo passo consiste em preparar esse classificador para execução dentro do ESP32. Diferentemente de ambientes como Python e scikit-learn, onde bibliotecas completas fazem todo o processamento automaticamente, o microcontrolador precisa de uma implementação explícita da função de decisão. Isso significa que o modelo treinado deve ser convertido para uma forma numérica simples, composta apenas pelos coeficientes do hiperplano e por um valor de bias, que serão utilizados diretamente no firmware.

O SVM Linear aprende uma equação da forma:

$$f(x) = w \cdot x + b$$

Em que:

- w representa o vetor de pesos aprendidos;
- b é o bias;
- x é o vetor de features extraído em tempo real pelo ESP32.

A inferência embarcada segue a regra:

- Se $f(x) \geq 0$ classifica como movimento.
- Se $f(x) < 0$ classifica como repouso.

Com isso, a complexidade computacional da inferência no microcontrolador reduz-se a operações de soma e multiplicação, perfeitamente adequadas ao processamento em tempo real.

Para extrair esses valores, utilizou-se o conjunto de treinamento já estruturado e o modelo SVM Linear treinado na etapa anterior. O código a seguir ilustra o processo utilizado no Jupyter Notebook.

```
import pandas as pd
from sklearn.svm import SVC

df = pd.read_csv("dados_rotulados_Principais_Features_Treino.csv")

X_train = df.drop(columns=["label"])
y_train = df["label"]

modelo = SVC(kernel="linear")
modelo.fit(X_train, y_train)

w = modelo.coef_[0]
b = modelo.intercept_[0]

print("Coeficientes (w):")
print(w)

print("\nBias (b):")
print(b)
```

Código-fonte 6 - Extração dos coeficientes e bias do SVM Linear
Fonte: Elaborado pelo autor (2025)

Os valores obtidos foram:

Coeficientes (w):

[-0.00165011 -0.00710379 -0.0144011 -0.00343501 -0.00081765 -0.04453173
0.00366878]

Bias (b):

1.8944849322991621

Esses coeficientes correspondem exatamente às features na ordem em que aparecem no arquivo “dados_rotulados_Principais_Features_Treino.csv”. No firmware, esse vetor será declarado como um array de ponto flutuante, permitindo que o ESP32 calcule a função de decisão a cada nova janela de dados recebida do sensor. O bias será representado por uma variável única. Esses valores serão incorporados ao código C, demonstrando como realizar a inferência embarcada com eficiência e baixo custo computacional.

A extração do hiperplano é uma etapa crítica no processo de implementação do modelo em sistemas embarcados, pois transforma o conhecimento aprendido no ambiente de treinamento em uma forma compatível com dispositivos de baixo consumo energético. Embora o dataset utilizado para esse estudo seja relativamente pequeno, sua finalidade é exclusivamente acadêmica e voltada à demonstração do fluxo completo de preparação de um classificador para uso em microcontroladores. Em aplicações reais, o ideal seria trabalhar com conjuntos de dados maiores e mais variados, de modo a aumentar a robustez do modelo e reduzir riscos de overfitting. Ainda assim, a metodologia apresentada aqui é totalmente válida e escalável para projetos reais.

2.7 Interpretação Matemática da Equação do Modelo

Uma vez extraídos os coeficientes e o bias do SVM Linear, é possível interpretar matematicamente como o modelo realiza a separação entre as classes repouso e movimento. Embora no ambiente embarcado a inferência seja reduzida a

operações aritméticas simples, compreender a equação que governa o classificador ajuda a visualizar o papel de cada feature no processo de decisão.

O SVM Linear define um hiperplano em um espaço multidimensional, onde cada dimensão corresponde a uma feature calculada anteriormente. No caso deste projeto, o vetor de entrada contém sete features derivadas do eixo Z do acelerômetro. O hiperplano aprendido pelo modelo divide o espaço em duas regiões: uma associada ao movimento e outra ao repouso. Os coeficientes determinam a inclinação e a orientação desse hiperplano, enquanto o bias desloca esse plano dentro do espaço das features.

Cada coeficiente indica o quanto uma feature contribui para empurrar uma amostra para um lado ou para o outro da fronteira de decisão. Coeficientes com maior magnitude influenciam mais fortemente o resultado, o que permite, por exemplo, identificar quais características do sinal são mais importantes para distinguir estados. O processo de classificação ocorre avaliando-se o sinal da soma ponderada das features com seus respectivos coeficientes, somada ao bias. Se o resultado for positivo, o modelo interpreta a janela como movimento; se for negativo, como repouso.

Essa interpretação também evidencia a razão pela qual modelos lineares são adequados para implementação embarcada: sua simplicidade matemática reduz consideravelmente o custo computacional no microcontrolador, ao mesmo tempo em que mantém boa capacidade de separação para problemas com classes bem definidas, como o caso analisado neste capítulo.

Essa etapa conclui o processo de tradução do modelo treinado para uma representação utilizável no firmware.

REFERÊNCIAS

EDGE IMPULSE. **End to End TinyML and Edge AI Workflow**. Edge Impulse Documentation, 2023. Disponível em: <https://docs.edgeimpulse.com/> Acesso em: 19 Jan. 2026.

KIM, M.; PARK, S.; KIM, H. A Survey on TinyML for Edge AI Applications. **Journal of KIISE**, [S. l.], 2022.

REDDY, Vishnu Vardhana Karna. A Comprehensive Review on Heart Disease Risk Prediction using Machine Learning and Deep Learning Algorithms. **Archives of Computational Methods in Engineering**, [S. l.], v. 32, 2024. DOI: 10.1007/s11831-024-10194-4.